Data Cleaning

```
library(dplyr)
library(readr)
```

Load Raw Data

```
raw <- read.csv("../data/Group3Data.csv", check.names = FALSE)

# Column names in the raw file start with a space (" timedelta")
names(raw) <- trimws(names(raw))

# Fix typo in the original dataset
raw <- raw |> rename(self_reference_avg_shares = self_reference_avg_sharess)

dim(raw)
```

```
## [1] 39644    61
```

Data Checks

Missing Values and Duplicates

```
sum(is.na(raw))
```

```
## [1] 0
```

```
sum(duplicated(raw$url))
```

```
## [1] 0
```

- No missing values and no duplicate articles

Articles With No Text

```
sum(raw$n_tokens_content == 0)
```

```
## [1] 1181
```

- 1,181 articles have 0 words, so the text never got scraped
- All their text features (sentiment, token length, rates) are 0
- Dropped

Impossible Word Rate

```
# n_unique_tokens is a share of words, so it can't be above 1
raw |>
  filter(n_unique_tokens > 1) |>
  select(n_tokens_content, n_unique_tokens, data_channel_is_tech)
```

```
##   n_tokens_content n_unique_tokens data_channel_is_tech
## 1             1570             701                    0
```

- One article, and every rate column for it is broken
- Not a tech article
- Dropped

Constant Column

```
# Stored as 0.9999999..., so round before counting
table(round(raw$n_non_stop_words, 6))
```

```
##
##     0     1  1042
##  1181 38462     1
```

- `n_non_stop_words` is 1 for every article with text, so it carries no information
- The one 1042 is the broken article above
- Column dropped

Keyword Placeholders

```
sum(raw$kw_min_min == -1)
```

```
## [1] 22980
```

```
sum(raw$kw_min_avg == 0)
```

```
## [1] 17102
```

- The `kw_` min columns are full of `-1` and `0` placeholders
- Kept, but we only plan to use `kw_avg_avg`
- `kw_avg_avg` = how many shares articles with the same keywords got in the past
- Kept as a control for topic popularity, not as something to recommend

Articles With No Channel

```
channel_cols <- grep("^data_channel_is_", names(raw), value = TRUE)
sum(rowSums(raw[channel_cols]) == 0)
```

```
## [1] 6134
```

- 6,134 articles aren't in any of the six channels
- Labeled "Other"

Publish Date

- The weekday columns are our only source for the timing question
- We check them against the date in each URL (`mashable.com/2013/01/07/...`)

```
day_names    <- c("Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun")
weekday_cols <- grep("^weekday_is_", names(raw), value = TRUE)

dated <- raw |>
  mutate(
    url_date   = as.Date(sub(".*mashable\\.com/(\\d{4}/\\d{2}/\\d{2}).*", "\\1", url),
                         format = "%Y/%m/%d"),
    data_day   = max.col(as.matrix(across(all_of(weekday_cols)))),  # 1 = Monday
    url_day    = as.integer(format(url_date, "%u")),                # 1 = Monday
    date_shift = (data_day - url_day) %% 7   # days the dataset is later than the URL
  )

table(date_shift = dated$date_shift)
```

```
## date_shift
##     0     1     2     3     4     5
## 30181  7059  1464   677   235    28
```

```
# timedelta is off by the same number of days (data was pulled Jan 8, 2015)
all(dated$url_date + dated$date_shift + dated$timedelta == as.Date("2015-01-08"))
```

```
## [1] TRUE
```

```
dated |>
  mutate(quarter = paste0(format(url_date, "%Y"), " Q",
                          (as.integer(format(url_date, "%m")) - 1) %/% 3 + 1)) |>
  group_by(quarter) |>
  summarise(articles = n(), pct_shifted = paste0(round(mean(date_shift > 0) * 100), "%")) |>
  knitr::kable()
```

quarter	articles	pct_shifted
2013 Q1	4654	0%
2013 Q2	4543	1%
2013 Q3	4640	2%
2013 Q4	4362	3%
2014 Q1	4523	4%
2014 Q2	5331	22%
2014 Q3	5736	44%
2014 Q4	5855	91%

- For 24% of articles the dataset's day is 1 to 5 days after the URL date
- `timedelta` moves with it, so both came from some other date
- Grows from almost none in early 2013 to nearly every article by late 2014
- We use the dataset's weekday and keep `url_weekday` for comparison

Clean Data

```
news <- dated |>
  filter(n_tokens_content > 0, n_unique_tokens <= 1) |>
  select(-n_non_stop_words, -data_day) |>
  mutate(
    channel = case_when(
      data_channel_is_tech          == 1 ~ "Tech",
      data_channel_is_lifestyle     == 1 ~ "Lifestyle",
      data_channel_is_entertainment == 1 ~ "Entertainment",
      data_channel_is_bus           == 1 ~ "Business",
      data_channel_is_socmed        == 1 ~ "Social media",
      data_channel_is_world         == 1 ~ "World",
      TRUE                               ~ "Other"
    ),

    # Day of week from the dataset's columns (main version)
    weekday = factor(day_names[max.col(as.matrix(across(all_of(weekday_cols))))],
                     levels = day_names),
    weekend = factor(if_else(is_weekend == 1, "Weekend", "Weekday"),
                     levels = c("Weekday", "Weekend")),

    # Day of week from the URL date (for comparison)
    url_weekday    = factor(day_names[url_day], levels = day_names),
    url_is_weekend = as.integer(url_day >= 6),

    # Calendar time, for checking seasonality
    year    = as.integer(format(url_date, "%Y")),
    month   = as.integer(format(url_date, "%m")),
    quarter = paste0(year, " Q", (month - 1) %/% 3 + 1),

    # Outcome
    log_shares = log(shares),

    # Groups used in plots
    img_group = cut(num_imgs, c(-Inf, 0, 1, 3, 10, Inf),
                    labels = c("0", "1", "2-3", "4-10", "11+")),
    vid_group = cut(num_videos, c(-Inf, 0, 1, Inf),
                    labels = c("0", "1", "2+")),
    length_group = cut(n_tokens_content, c(0, 250, 500, 1000, 2000, Inf),
                       labels = c("<250", "250-500", "500-1k", "1k-2k", "2k+")),
    title_tone = case_when(
      title_sentiment_polarity < 0 ~ "Negative",
      title_sentiment_polarity > 0 ~ "Positive",
      TRUE                         ~ "Neutral"
    ) |> factor(levels = c("Negative", "Neutral", "Positive"))
  ) |>
  select(-url_day)

tech <- news |> filter(data_channel_is_tech == 1)
```

```r
tibble(
  step = c("Raw articles", "Dropped: 0 words", "Dropped: bad rate",
           "Clean articles", "Clean tech articles"),
  articles = c(nrow(raw),
               -sum(raw$n_tokens_content == 0),
               -sum(raw$n_tokens_content > 0 & raw$n_unique_tokens > 1),
               nrow(news), nrow(tech))
) |> knitr::kable(format.args = list(big.mark = ","))
```

step	articles
Raw articles	39,644
Dropped: 0 words	-1,181
Dropped: bad rate	-1
Clean articles	38,462
Clean tech articles	7,325

```r
count(news, channel, sort = TRUE) |>
  mutate(pct = paste0(round(n / sum(n) * 100, 1), "%")) |>
  knitr::kable(format.args = list(big.mark = ","))
```

channel	n	pct
World	8,168	21.2%
Tech	7,325	19%
Entertainment	6,855	17.8%
Business	6,235	16.2%
Other	5,491	14.3%
Social media	2,311	6%
Lifestyle	2,077	5.4%

Save

```r
dir.create("../data/processed", showWarnings = FALSE, recursive = TRUE)

# .rds keeps column types and factor order. The .csv copies are for viewing
saveRDS(news, "../data/processed/news_clean.rds")
saveRDS(tech, "../data/processed/tech_clean.rds")

write_csv(news, "../data/processed/news_clean.csv")
write_csv(tech, "../data/processed/tech_clean.csv")
```

New Columns

Column	Meaning
`url_date`	Publish date taken from the article URL
`date_shift`	Days the dataset's weekday falls after the URL date (0 for most articles)
`channel`	Article channel as one label: Tech, Business, Entertainment, Lifestyle, Social media, World, or Other
`weekday`	Day of the week published (Mon to Sun), from the dataset's weekday columns
`weekend`	"Weekday" or "Weekend" label, from `is_weekend`
`url_weekday`	Day of the week published (Mon to Sun), from the URL date
`url_is_weekend`	1 if the URL date falls on a Saturday or Sunday
`year`	Year published (2013 or 2014), from the URL date
`month`	Month published (1 to 12), from the URL date
`quarter`	Quarter published, such as "2014 Q3", from the URL date
`log_shares`	Natural log of shares, used as the model outcome
`img_group`	Number of images, grouped: 0, 1, 2-3, 4-10, 11+
`vid_group`	Number of videos, grouped: 0, 1, 2+
`length_group`	Word count, grouped: under 250, 250-500, 500-1000, 1000-2000, 2000+
`title_tone`	Headline tone: Negative (below 0), Neutral (exactly 0, no sentiment words), or Positive (above 0)

Other

- Removed `n_non_stop_words`
- Renamed `self_reference_avg_sharess` to `self_reference_avg_shares`